Contribution Margin Recompute & Caching

An Engineering Guide — Background Tasks, Polling, Locks & Caching

Ad Genius · Unit Economics

Branch: `recompute-cm-on-edit` · Generated 9 June 2026
Frontend: Next.js 15 on Vercel · Backend: Python + Celery on AWS · PostgreSQL · Redis

1. What We Built (Overview)

This work delivered **two related features** on the Unit Economics admin page.

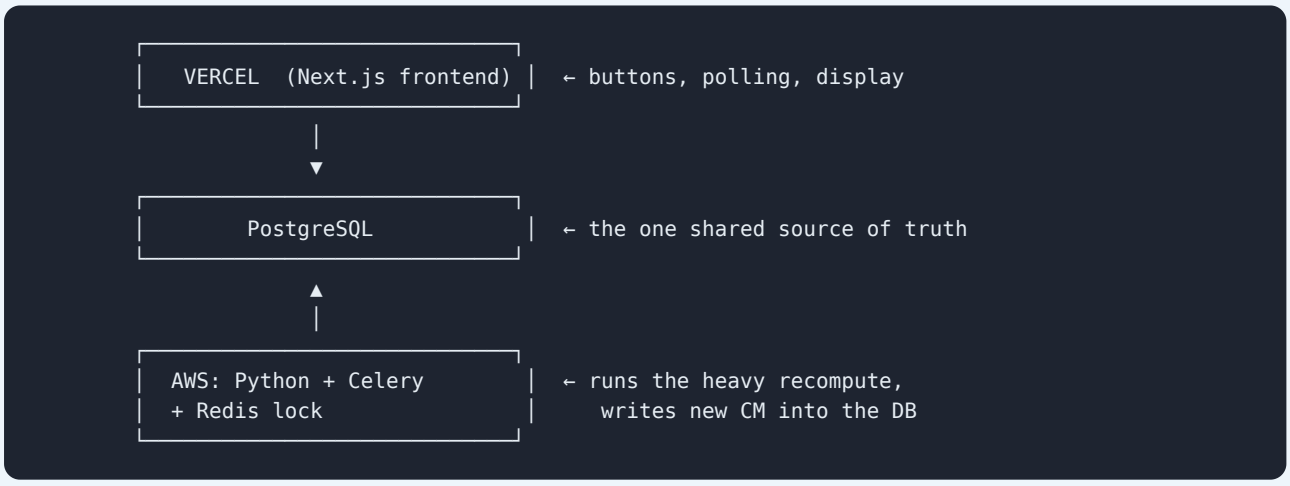
Feature 1 — The "Recompute CM" button

A button that recalculates contribution margin (CM) in the background after you edit cost values, with safety guards so data never gets corrupted mid-calculation.

Feature 2 — Cache removal (always-fresh numbers)

Removed a stale 5-minute in-memory config cache that caused old CM numbers to show until the server was restarted. Reads are now always fresh from the database.

The architecture at a glance



2. Feature 1 — The Recompute CM Flow

The contribution-margin calculation is a PostgreSQL stored procedure that can take several minutes — far longer than an HTTP request is allowed to stay open. So it runs as a **background job**, and the browser checks on it periodically.

The full journey of one click

```
1. You edit costs        → saved to PostgreSQL
2. "Recompute CM" appears → click it
3. Next.js  → forwards to → Python (AWS)
4. Python   → acquires a Redis lock (so only one runs)
            → queues a Celery background task
            → returns immediately (does NOT wait)
5. Celery   → runs the stored procedure → writes new CM to PostgreSQL
            → releases the lock when done
6. Browser  → polls every 5 seconds: "is it done yet?"
            → spinner while running → success toast when finished
```

Why background + polling instead of waiting?

Approach	Problem
Make the browser wait for the result	The calculation takes minutes; the HTTP connection times out and dies. ❌
Background job + polling	Browser fires the job, then asks "done yet?" every 5s. No timeout. ✅

3. What Is Polling? (Plain English)

Polling = repeatedly asking "are you done yet?" at a fixed interval until you get an answer.

> **Analogy — baking a cake** 🎂
> You put a cake in the oven (start the job). You don't stand frozen staring at it. Instead, every 5 minutes you open the oven and check: "ready?" — "not yet" — "not yet" — "done!" That repeated checking *is* polling.

How our polling works

```
Every 5 seconds, the browser calls /status:

  running: true   → keep the spinner, keep polling
  success: true   → done!  clear the edited-state, show success toast
  success: false  → it failed → show error, keep your edits so you can retry
```

What happens if the status check itself fails?

If the browser loses connection and a status check fails **3 times in a row**, we stop polling — but we do **NOT** pretend the job finished.

> **This is the `recomputeStatusUnknown` safety flag.**
> A failed status check means "*I don't know* what's happening" — not "it's finished". So editing stays **locked**, and a message tells you to press **Refresh** to re-check the real status.

Why this matters

	Without the flag (wrong)	With the flag (correct)
Connection lost	"Must be done" → unlock editing	"Don't know" → stay locked
But job still running…	You edit → corrupts the result ❌	You wait / press Refresh ✅

4. The Lock (Preventing Duplicates & Bad Edits)

A **Redis lock** is a flag in shared memory that means "a recompute is currently happening." It does two jobs:

1. **Stops two recomputes** running at the same time.
2. **Blocks edits** to cost values while a recompute is reading them.

Why block edits during a recompute?

> If the recompute is reading the cost values *while you change them*, it reads a mix of old + new numbers → **wrong results**. The lock prevents this.

The "is the lock holder dead?" check

If the Python server **crashes** mid-job, the lock is never released and would block everyone for up to 90 minutes. So before rejecting a new request, we check whether the task holding the lock is actually still alive:

```
_lock_holder_is_dead():
  - no owner on record           → stale, safe to clear  (True)
  - task state SUCCESS/FAILURE/REVOKED → finished/dead, safe to clear (True)
  - task state PENDING/STARTED/RETRY   → still alive, DO NOT clear   (False)
```

The decision tree when you click Recompute

```
Try to acquire the lock
   │
   ├─ got it          → start the recompute ✅
   │
   └─ already held    → is the holder dead?
                          ├─ YES → clear stale lock, try again
                          └─ NO  → block with 409 "already running"
```

5. The Three Safety Fixes (P1 / P2 / P2)

A code review surfaced three real risks, all now fixed.

P1 Save guard: block when unsure

Before saving, the app asks Python "is a recompute running?" We now split the failure cases:

Situation	Meaning	Decision
Python answers with an error	Python is **up** → maybe running	Block (503)
Can't connect at all	Python is **down** → nothing running	Allow (fail-open)

P2-backend No duplicate recompute

The task id is now generated first and stored *inside* the lock atomically, so the lock can never exist "without an owner" — closing the race where a valid lock was mistaken for stale and deleted.

P2-frontend

Lost connection ≠ finished

After 3 failed status checks, editing stays **locked** (status unknown) instead of unlocking. The Refresh button re-checks the real status to recover.

6. What Is Caching? (Plain English)

Caching = keeping a temporary copy of something so you don't have to fetch it from the slow source every time.

> **Analogy — fridge vs grocery store** 🥛
> The grocery store is the database (slow, far). Your fridge is the cache (fast, nearby). You keep milk in the fridge so you don't drive to the store every time you want a glass. But if the milk goes *old* in the fridge, you drink old milk — that is a "stale cache".

Server-side vs client-side cache

	Server-side cache	Client-side cache
Lives in	The Next.js server's memory (on Vercel)	Your browser
Our config cache was…	✅ here (server-side)	not here
Cleared by a browser refresh (F5)?	❌ No — F5 doesn't touch server memory	sometimes

7. The Caching Bug We Fixed

What went wrong

```
You edit costs → save to DB ✅
You view CM    → Next.js shows the value computed from its
                 OLD cached config (up to 5 minutes) ❌
                 → you saw stale numbers until the server restarted
```

> **Root cause:** the config was stored in the server's memory for 5 minutes (`configCache`), and the function meant to clear it (`clearConfigCache()`) was **never called anywhere** — dead code. Restarting the server wiped the memory, which is why that "fixed" it temporarily.

Why "just clear the cache on save/refresh" did NOT work on Vercel

Vercel runs your app as **serverless functions** — it can run **several copies** at once, and shuts idle copies down (cold starts wipe memory). Each copy has its **own separate memory**.

```
Copy A → cache cleared ✅   ← your click landed here
Copy B → still old     ❌   ← untouched
Copy C → still old     ❌

Next page load hits Copy B → you STILL see old numbers ❌
```

So clearing the in-memory cache only ever fixes one copy out of many. It cannot reliably solve the problem on Vercel.

8. What We Did — The Fix

> **We removed the cross-request cache and now read fresh from the database, with per-request memoization via React's `cache()`.**

The change (in `cm-enrichment.ts`)

- Deleted the 5-minute in-memory `configCache` `Map`.
- Deleted `clearConfigCache()` (it was unused / dead code).
- Wrapped the DB read in React's `cache()` so it loads **once per request**.
- Kept the function signature identical → **no caller breaks**.

What React `cache()` does

```
WITHIN one page request:
  KPI cards → reads config from DB  (1 query)
  Table     → reuses it ✅ (no extra query)
  Totals    → reuses it ✅ (no extra query)

ACROSS requests:
  nothing is kept → every new request reads FRESH → never stale
```

> **Best of both worlds:** the *speed* of caching *inside* a request, without the *staleness* of caching *across* requests.

Will it break anything? — No

Check	Result
Function signature	Unchanged → all 4 callers still work
`forceRefresh` argument	Every caller passed `false` → ignoring it changes nothing
`clearConfigCache()` removed	Was never called → safe
CM calculation logic	Untouched
Lint	✅ Passed
Build	⚠️ Run `npm run build:without-tests` to confirm

9. Is It Best Practice for Production?

Yes — it's a solid, production-safe, correctness-first choice. Removing a mutable in-memory cache on serverless is the right call, and React `cache()` is the idiomatic Next.js pattern.

> **The honest caveat:** we now read the (tiny) config tables on every request. Fine at normal traffic; at very high traffic this adds database load. The "gold standard" at scale is a *shared cache with invalidation*.

10. The Upgrade Path — `revalidateTag`

When traffic grows, the fuller best-practice pattern gives you **both speed and freshness**.

The idea

Cache the config in Vercel's **shared** store with a name tag (e.g. `"unit-economics-config"`). When you save, call `revalidateTag("unit-economics-config")` — which clears that tag on **every** server at once.

```
You save new costs
   → revalidateTag("unit-economics-config")
   → ALL Vercel copies drop the old config together ✅
   → next read fetches fresh from DB and re-caches
```

Comparison

	In-memory (old, broken)	Read fresh (now)	revalidateTag (at scale)
Always correct	❌	✅	✅
Fast (cached)	✅ (but stale)	⚠️ reads each request	✅
Works across all Vercel copies	❌	✅	✅
Extra complexity	—	none	Map→JSON conversion needed

> **Gotcha for the upgrade:** the config uses JavaScript `Map`s, which don't survive being saved as JSON in the shared cache. You'd convert them to plain arrays/objects before caching and back after reading.

11. Glossary

Term	Plain-English meaning
Polling	Repeatedly asking "are you done yet?" on an interval (every 5s here).
Background job	Work that runs separately so the browser doesn't have to wait.
Celery	The Python tool that runs background jobs.
Redis lock	A shared flag meaning "a recompute is running" — prevents duplicates and bad edits.
TTL	Time To Live — how long something is kept before it expires.
Cache	A temporary fast copy to avoid hitting the slow source every time.
Stale data	An old cached copy that no longer matches the real (database) value.
Server-side	Runs on the server (Vercel/AWS), not in your browser.
Serverless	Vercel runs many short-lived copies of your app on demand; no fixed server.
Cold start	A fresh copy of the app starting up with empty memory.
Fail-open	When unsure, allow the action (used only when Python is truly down).
Fail-closed	When unsure, block the action (used when Python is up but errored).
React `cache()`	Memoizes a function for one request, then forgets it — fast within a request, fresh across requests.
`revalidateTag`	Clears a named shared cache on all servers at once.
Memoization	Remembering a function's result so repeat calls reuse it instead of recomputing.

12. Files Touched

File	Role
`cm_recompute_tasks.py`	Celery task that runs the stored procedure (Python/AWS)
`cm_recompute.py`	Python trigger + status endpoints, Redis lock logic
`recompute-cm/route.ts`	Next.js proxy that forwards the trigger to Python
`recompute-cm/status/route.ts`	Next.js proxy for the status check
`cm-recompute-lock.ts`	Blocks edits during a recompute (the save guard, P1 fix)
`page.tsx`	Button, dirty tracking, polling, status-unknown state, toasts
`cm-enrichment.ts`	Config loading — cache removed, per-request memoization added

---

End of guide. Generated from the working session on the `recompute-cm-on-edit` branch. For the latest code, see the commits referenced on the cover page.